

HotpotQA Benchmark Report

AgentOpt Model Selection Framework

9 Bedrock Models | 199 Samples | 2-Tuple (Planner+Solver) | March 2026

1. Brute Force Ground Truth (Planner + Solver)

- Exhaustive evaluation: all 81 combos (9x9) x 199 HotpotQA samples.
- 2-tuple architecture: planner proposes steps, solver uses search tools.
- Best combo: Ministral 3 8B + Claude Opus 4.6 at 74.78%.
- Total brute force cost across all combos: \$51.48.
- Total evaluations: 16,108 of 16,119 (some samples skipped due to API errors).

Top 15 Combinations (of 81)

Rank	Planner + Solver	Accuracy	Cost
1	Ministral 3 8B + Claude Opus 4.6	74.78%	\$2.71
2	Qwen3 32B + Claude Opus 4.6	72.97%	\$2.67
3	Claude 3 Haiku + Claude Opus 4.6	72.58%	\$2.67
4	Qwen3 Next 80B A3B + Claude Opus 4.6	72.32%	\$2.66
5	Qwen3 Next 80B A3B + gpt-oss-120b	71.29%	\$0.13
6	Kimi K2.5 + Claude Opus 4.6	71.09%	\$2.53
7	Qwen3 32B + gpt-oss-120b	70.47%	\$0.12
8	Qwen3 32B + Qwen3 Next 80B A3B	69.12%	\$0.11
9	Claude 3 Haiku + Qwen3 Next 80B A3B	68.60%	\$0.15
10	Ministral 3 8B + gpt-oss-120b	68.56%	\$0.12
11	Claude 3 Haiku + gpt-oss-120b	68.44%	\$0.16
12	Kimi K2.5 + Qwen3 Next 80B A3B	68.41%	\$0.28
13	Qwen3 Next 80B A3B + Qwen3 Next 80B A3B	68.29%	\$0.12
14	Qwen3 32B + gpt-oss-20b	67.75%	\$0.09
15	Ministral 3 8B + Qwen3 Next 80B A3B	67.46%	\$0.11

Bottom 12 Combinations (of 81)

Rank	Planner + Solver	Accuracy	Cost
70	Claude Opus 4.6 + Claude Opus 4.6	32.70%	\$2.00
71	Claude Opus 4.6 + Kimi K2.5	32.44%	\$2.01
72	Claude Opus 4.6 + Qwen3 Next 80B A3B	32.05%	\$2.01
73	Claude Opus 4.6 + gpt-oss-120b	32.00%	\$2.01
74	Claude Opus 4.6 + Ministral 3 8B	31.80%	\$2.01
75	Claude Opus 4.6 + Claude 3 Haiku	31.80%	\$2.01
76	Claude Opus 4.6 + Qwen3 32B	31.52%	\$2.01
77	Claude Opus 4.6 + gpt-oss-20b	31.31%	\$2.01
78	Qwen3 Next 80B A3B + Claude Haiku 4.5	30.85%	\$0.71
79	Claude Opus 4.6 + Claude Haiku 4.5	30.81%	\$2.01

80	Claude Haiku 4.5 + Claude Haiku 4.5	26.57%	\$0.79
81	Qwen3 32B + Claude Haiku 4.5	25.11%	\$0.72

Note: Per-sample latency not reported. HotpotQA ran 81 combos concurrently, inflating per-sample times to 600-900s (API-side queueing). GPQA and BFCL latencies are reliable because they are 1-tuple (9 models, minimal cross-model contention).

Note: Ranks 70-77 are ALL Opus-as-planner combos (31-33%), regardless of solver.

The solver barely matters when Opus plans -- because Opus terminates immediately (see Section 4).

2. Offline Selector Simulation (50 Seeds)

- Per-sample scores from brute force replayed with each selector's logic
-- zero additional API calls required.
- Each selector simulated 50 times with different random seeds.
- Hill climbing uses topology-aware neighbors (quality/speed ranking).
HC (1) = 1 random restart; HC (3) = 3 restarts (library default).
- Metrics: Find Rate = % finding true best. Mean Acc = avg accuracy of chosen combo.

Selector	Found Best	Mean Acc	Evaluations	Cost	Savings
Brute Force	100%	74.78%	16,108	\$51.48	--
Arm Elimination	90%	74.12%	4,654	\$18.49	64%
Hill Climbing (3)	44%	73.38%	5,031	\$19.21	63%
Bayesian Opt.	8%	72.78%	3,979	\$12.13	76%
Random Search	30%	72.34%	4,176	\$13.26	74%
Epsilon LUCB	14%	69.96%	477	\$1.86	96%
Hill Climbing (1)	24%	68.54%	1,881	\$8.35	84%
Threshold SE	2%	63.62%	1,926	\$3.50	93%
LM Proposal	0%	34.41%	199	\$1.86	96%

3. Key Takeaways

Brute Force Insights

- Best combo is counter-intuitive: cheapest planner (Ministral 8B) + best solver (Opus).
- Opus as solver dominates: top 6 combos all use Opus as solver (71-75%).
- Planner quality is secondary: Ministral, Qwen3 32B, C3 Haiku all achieve ~73% with Opus.
- Opus+Opus is worst Opus-solver combo (32.70%) -- see Section 4 for analysis.
- gpt-oss-120b is the best non-Opus solver: 68-71% at ~\$0.20 (vs ~\$2.70 for Opus).
- Best value combo: Qwen3 Next + gpt-oss-120b at 71.29% for only \$0.13 --
21x cheaper than the best combo (74.78%, \$2.71) with only a 3.5% accuracy drop.

Selector Insights

- Arm Elimination is again the best selector: 90% find rate at 64% cost savings.
Consistent winner across 1-tuple (GPQA: 98%) and 2-tuple (HotpotQA: 90%).
- Hill Climbing (3 restarts) now works for 2-tuple: 44% find rate, 63% savings.
Topology-aware neighbors make it viable (was degenerate with naive neighbors).
- Bayesian Opt. improved on larger space: very consistent accuracy (72.6-74.8% range).
Low find rate (8%) but always picks a near-optimal combo.
- Epsilon LUCB is ultra-cheap (\$1.86, 96% savings) -- best budget option.
- LM Proposal fails (0% find rate, 34% mean acc): GPT-4.1 picks varying combos

across seeds (range 32.7-54.1%), all poor. Cannot predict the counter-intuitive best combo (cheapest planner + best solver) from task previews alone.

4. Capability as Liability: Why Opus Fails as Planner

- The best combo (Ministral+Opus, 74.78%) uses the weakest model as planner.
- The worst Opus-solver combo (Opus+Opus, 32.70%) uses the strongest as planner.
- Head-to-head (excluding 83 ties): Ministral+Opus wins 115; Opus+Opus wins only 1.
- The 1 sample Opus+Opus wins: 'Which was fought earlier, the Seven Days Battles or the Battle of Manila?' -- Civil War 1862 vs WWII 1945, trivial for Opus.
Opus planner answers correctly from parametric knowledge (2,068 tokens, score=1.0).
Meanwhile MO's solver uses heavy tool search (6,201 solver tokens) but gets only a partial F1 match (0.26) -- search noise degrades the answer format.
35/81 combos score 0.0 on this sample -- it's actually hard for most combos despite the 'easy' question, likely due to answer format issues.
This is the only sample where both conditions hold: (1) Opus knows the answer from memory, AND (2) tool-augmented search produces a worse result.
Most HotpotQA questions are obscure enough that condition 1 fails.

The Evidence: Token Analysis

Metric	Ministral+Opus	Opus+Opus
Accuracy	74.78%	32.70%
Avg input tokens/sample	4,058	1,575
Est. planner input/sample	1,479	~1,479 (same prompt)
Est. solver input/sample	2,580	~96
Solver input ratio	1.0x (baseline)	0.04x
Low-input samples (<2K tok)	0/199 (0%)	182/199 (91%)

The Planner Prompt (verbatim)

```
SYSTEM: You are a planner in a planner/solver QA loop.  
Given context, question, and prior solver output, propose ONE concrete next step.  
If enough evidence is available, return terminate() and include a concise final answer.  
Format:  
- Ongoing: NEXT: <single concrete step>  
- Finish: terminate() FINAL: <answer>  
  
USER: {question + 10 context paragraphs}  
No prior solver output yet. Propose the first concrete next step.
```

Why terminate() Exists (Standard Agentic Design)

- Early stopping is standard practice in agentic systems (ReAct, Reflexion, LATS).
- Without it, the agent loops indefinitely -- wasting tokens and cost on solved tasks.
- The planner must have a way to signal 'I have enough information to answer.'
- This is not a prompt engineering mistake -- it is a necessary control mechanism.
- The problem is that Opus exploits the escape hatch when it should not.

The Mechanism: Premature Termination

- HotpotQA provides 10 context paragraphs per question, only 2 of which contain the required supporting facts.

- Opus sees the context and calls terminate() immediately on ~91% of samples, returning a FINAL answer from the planner. The solver is never invoked.
- Estimated solver input is only ~96 tokens/sample (vs 2,580 for Ministral+Opus).
- Without search tools, Opus attempts multi-hop reasoning from raw context alone -- and fails ~67% of the time because it cannot reliably filter distractors.

Why Ministral Succeeds

- Ministral (8B params) lacks the parametric knowledge to attempt multi-hop reasoning.
- It correctly outputs 'NEXT: search for ...' instead of terminate(), which invokes the solver (Opus) with search tools (wikipedia, web search).
- The solver uses tools to retrieve evidence and produces accurate answers (74.78%).
- The weaker planner delegates to the tool-augmented solver as the architecture intended.

The Takeaway: Capability as Liability

- Model capability can be a liability in agentic architectures.
- Opus is 'too smart' for the planner role: it bypasses the tool-augmented solver because it believes it already has enough evidence to answer directly.
- This is a concrete example of how stronger models can underperform weaker ones when the architecture is designed to leverage external tools.
- Implication for AgentOpt: model selection must consider architectural fit, not just raw model quality. The optimal model depends on the role.
- This finding is only discoverable through exhaustive 2-tuple search -- a single-model benchmark would always rank Opus first.